SFT done cheaply

What supervised fine-tuning can and cannot teach, how little data it needs, why LoRA is now the default and when it is not, and the arithmetic that puts a 7B post-training run on one GPU

The base model is done. It has read two trillion tokens, it completes Spanish legal text as fluently as English code, and when you type "¿Cuál es la capital de Perú?" it answers with three more questions about South American capitals, because that is what the web does after a question. The gap between that model and one a user can talk to is the *format gap* of Chapter 27, and closing it is supervised fine-tuning: show the model a few thousand to a few hundred thousand conversations and train it to produce the assistant's turns. The expensive version of this is well known — a full-parameter run over a million examples on a cluster — and most teams still budget for it. The cheap version is what this chapter is about, and it rests on three results from 2023–2025 that are now settled: SFT needs far less data than people use (a thousand curated examples aligned a 65B model; a thousand reasoning traces made a 32B model beat o1-preview on competition math[12]); low-rank adapters match full fine-tuning for almost every SFT dataset that fits in them, at a third less compute and a tenth of the memory, provided they are applied to the right layers at the right learning rate[3]; and most of what goes wrong in SFT is not in the optimizer but in the data and the loss mask. A 7B SFT run that follows this chapter costs a few GPU-hours and fits on one 80GB card — or on a 24GB one with quantization.

The chapter assumes Chapter 11 (which examples are worth training on) and Chapter 27 (where the examples come from), and it hands off to Chapter 25: SFT is the stage that gives the model a format and a starting policy; the capabilities that need exploration are taught afterwards with rewards.

23.1 What SFT is for — and what it is not

Start with what changes inside the model during SFT, because the budget follows from it. The LIMA experiment of 2023 fine-tuned a 65B base model on exactly 1,000 carefully written prompt–response pairs, no preference data, no RL, and found its outputs equivalent to or preferred over GPT-4's in 43% of human comparisons[1]. The authors' explanation — the *superficial alignment hypothesis* — is that a base model already contains the knowledge and most of the skills; SFT teaches it the *format* of an answer and *which sub-distribution* of its pre-training to use when a user speaks. That is a small amount of information, which is why a small amount of data carries it, and why the Chapter 11 selection results keep finding that 5–10% of a typical SFT set is as good as the whole: the rest teaches what the model already does.

Two refinements since then set the boundaries. First, SFT *can* teach a skill when the skill is visible in the text: long reasoning traces are the example. Fine-tuning Qwen2.5-32B-Instruct on the 1,000 traces of s1K — selected for difficulty, diversity, and quality — produced a model that beat o1-preview on MATH and AIME24 by up to 27%, with the training run taking well under an hour on 16 H100s[2]; scaled up, the 1.2M traces of OpenThoughts3 took a 7B model to 53% on AIME 2025[4]. What those traces teach is a *behavior* — think longer, check, backtrack — that the base could already perform in pieces, and the data makes it the default. Second, SFT *memorizes* where RL generalizes: on tasks with controlled distribution shifts (an arithmetic card game with rule variants, a visual navigation task), SFT-trained models tracked their training data and failed out of distribution, while RL-trained ones transferred — but the same study found that SFT was still needed *first*, to stabilize the output format so that RL had something to optimize[5]. Figure 23.1 draws the division of labor that follows.

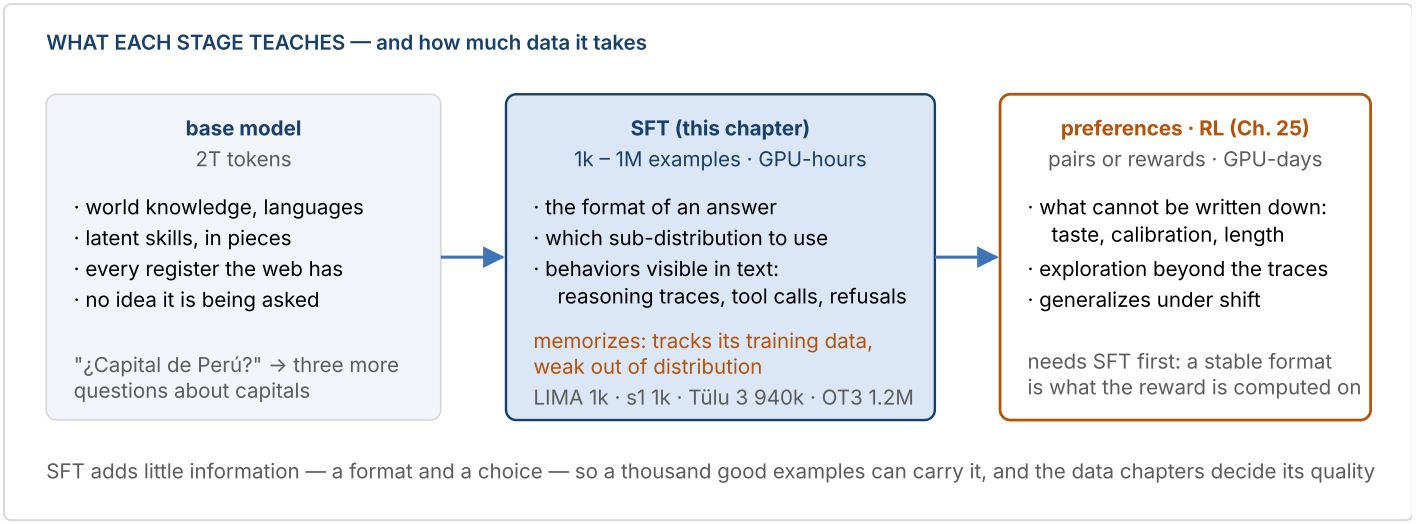


Figure 23.1 — SFT adds a format and a choice, not knowledge. The base holds the capabilities; SFT selects and shapes them with little data; what cannot be demonstrated in text is left to preference optimization and RL, which need SFT's stable format to work on. The budget for each stage follows from what it teaches.

The practical consequence is the chapter's thesis. Because SFT carries little information, its cost is dominated by things you control — how many examples, how long they are, how many epochs, how many parameters you update — and not by the model's size. A 7B model and a 70B model need about the same SFT *data*; they differ in the cost per token, which is where the parameter-efficient methods of §23.6 enter.

23.2 The data: small, hard, new — and correctly masked

Chapter 11 gave the selection rules and this section applies them, with the one mechanical detail that ruins more SFT runs than any hyperparameter. **How much.** The public recipes span three orders of magnitude and the spread is informative. LIMA's 1,000 and s1's 1,000 are the floor for teaching a format or a single behavior to a strong base; Deita's 6,000 selected for complexity, quality, and diversity matched models trained on ten times more[6]; Tülu 3's SFT mixture is about 940K prompts, because it targets a dozen skills at once (math, code, precise instruction following, safety, multilingual chat) and each skill wants its own tens of thousands[7]; OpenThoughts3's 1.2M is large because reasoning traces are long and the gains kept coming with scale[4]. The rule that reconciles

them: budget by *skills* × *examples per skill*, not by a total, and run Chapter 11's IFD and pass-rate filters so that what remains is hard and new to *your* base — a downloaded set is often half already-known.

Which shape. SFT data is conversations in a chat template: system, user, and assistant turns with special tokens marking each. The template is part of the model from now on — the same template at training and inference, or the model is being asked questions in a dialect it never saw — and the tokenizer's chat template (Chapter 12) is where it lives. Multi-turn examples teach the model to track context; include them, and include turns where the assistant asks a clarifying question or declines, because the model learns the *distribution* of assistant behavior, not only its best case.

Which tokens. Here is the detail. The loss in SFT is the same next-token cross-entropy as pre-training, but it should be computed *only on the assistant's tokens*: the user's turn and the system prompt are context, not targets. Training on them too — the default in many scripts, because it is the pre-training default — spends a third or more of the gradient on learning to predict what users type, teaches the model to continue a conversation by writing the next user message, and dilutes the signal from the answers. Figure 23.2 shows the mask; the code below is how the Hugging Face stack applies it.

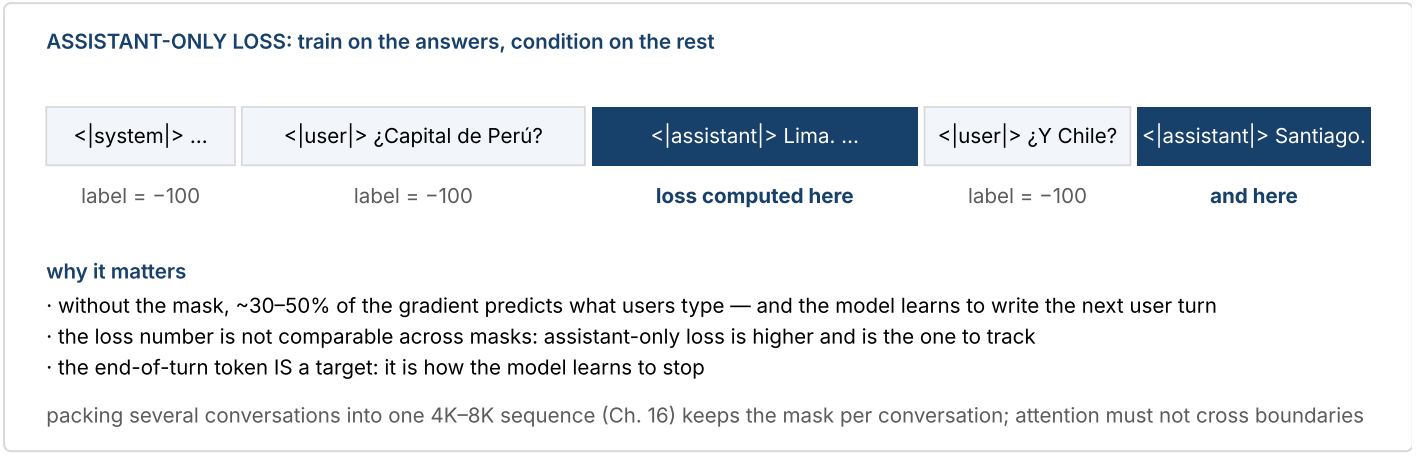


Figure 23.2 — The loss mask is the SFT objective. Only assistant tokens (and the end-of-turn token that teaches the model to stop) are targets; everything else is conditioned on with its label set to −100 so the cross-entropy ignores it.

```
# SFT with the Hugging Face stack: chat template, assistant-only loss, packing (TRL)
from datasets import load_dataset
from trl import SFTConfig, SFTTrainer
ds = load_dataset("HuggingFaceH4/ultrachat_200k", split="train_sft")   # "messages"
ds = ds.filter(keep_hard_and_new)   # Ch. 11: IFD above median, not "seen", decontami
cfg = SFTConfig(
    output_dir="sft-7b", max_length=4096,
    assistant_only_loss=True,          # labels = -100 outside assistant turns
    packing=True, packing_strategy="bfd",      # Ch. 16: no cross-conversation attent
    per_device_train_batch_size=4, gradient_accumulation_steps=8,   # effective batch
    learning_rate=1e-5, lr_scheduler_type="cosine", warmup_ratio=0.03,
    num_train_epochs=2, bf16=True, gradient_checkpointing=True,
    logging_steps=10, save_strategy="epoch", report_to="wandb")
trainer = SFTTrainer(model="meta-llama/Llama-3.1-8B",   # base model + its chat templ
                     train_dataset=ds, args=cfg)
trainer.train()
# before the long run: decode one batch of labels — only assistant text + end-of-turn
```

Two checks belong in every pipeline before the long run. Decode one batch's labels and read them: only assistant text and end-of-turn tokens should survive the mask (a template without generation markers silently trains on everything). And count: assistant tokens over total tokens tells you the effective training set — a dataset of long prompts and short answers can be 20% targets, which changes both the budget and the loss you should expect.

23.3 What SFT is for: the goals, and the data each one needs

"Format and sub-distribution" is the mechanism; the engineer arriving at this chapter has a list of behaviors to install, and the useful question is what data each behavior takes. The goals that production SFT sets are built from are a short list, and they differ in the one dimension that sets the budget: how much of the behavior is already latent in the base, versus how much the examples must carry. Table 23.1 gives the list with the data type, the order of magnitude, and the public set that is the usual starting point; the sizes are what the recipes in §23.9 actually used, not theory.

Table 23.1 — Goals of SFT and what each costs in data. Example counts are the ranges used by Tülu 3, SmolTalk, LIMA/s1, and the Llama 3 and Qwen3 reports; the public set is a starting point to audit with Chapter 11, not a recommendation to use whole.

GOAL	WHAT THE EXAMPLES LOOK LIKE	HOW MANY	PUBLIC STARTING POINT	NOTE
Answer format and register	Prompt → one well-written answer	1K–10K	LIMA, No Robots, OpenAssistant	The cheapest goal; quality beats count by a wide margin
Multi-turn chat	3–8 turn dialogues, including clarifying questions	50K–200K	UltraChat-200k, WildChat, Smol-Magpie-Ultra	The model learns the *distribution* of assistant behavior; keep the imperfect turns
Precise instruction following	Prompts with verifiable constraints (length, format, forbidden words) and compliant answers	10K–30K	Tülu 3 Persona IF, Smol-constraints	Measured directly by IFEval; the goal with the best data-to-score ratio
Structured output	Schema in the prompt → valid JSON / table / code block	5K–30K	Smol-rewrite, Smol-summarize, self-built from your schemas	Validate every target with a parser before training on it
Tool and function calling	Tool specs in the system turn; assistant emits a call; tool result; final answer	20K–80K	APIGen / xLAM function-calling, Glaive, ToolACE	Needs the template's tool tokens; evaluate on BFCL; most failures are format, not reasoning
Math and code	Problem → worked solution; task → program; verified by answer or execution	100K–300K	NuminaMath, MetaMathQA, Evol-CodeAlpaca, OpenCodeReasoning	Rejection-sample from your own model where you can (Ch. 27); verify every target
Long reasoning traces	Problem → 5–20K-token trace → answer	1K (one strong base) – 1M (scale)	s1K, OpenThoughts3	Long context, several epochs; the one goal where SFT installs a new default behavior
Safety and refusals	Harmful request → refusal; borderline request → helpful answer	5K–50K, *paired*	WildGuardMix, CoCoNot, WildJailbreak	§23.4: over-refusal is the failure mode; always pair with compliance data

GOAL	WHAT THE EXAMPLES LOOK LIKE	HOW MANY	PUBLIC STARTING POINT	NOTE
Language and locale	In-language versions of all of the above	20–40% of the mix for a Spanish-first model	Aya, in-language synthesis (Ch. 27)	Translation is a fallback; register and refusal norms differ by language
System-prompt obedience	Varied system prompts (persona, rules) + conversations that honor them	10K–30K	SystemChats, self-built	Without it the model ignores the system turn; cheap and often forgotten

Three observations organize the table. The goals that are *already in the base* — format, register, chat — take the least data and the most curation; the goals that are *verifiable* — instruction constraints, structured output, math, code, tools — scale with data because every target can be checked, and rejection sampling (Chapter 27) manufactures them; and the goals that are *norms* rather than skills — safety, locale, system-prompt obedience — are small in count but fragile in balance, because the model generalizes the norm from a few thousand examples and gets the boundary wrong in whichever direction the data leaned. The next section, on safety, is the clearest case. Each goal also comes with its own evaluation (IFEval for constraints, BFCL for tools, a parser for structured output, the Chapter 9 suite for math and code), and the mix is tuned goal by goal against those numbers, never against the training loss.

A corollary that saves money: a goal is only worth its rows if the base model fails at it. Run the evaluation on the *base* with a minimal template first — a 2025 base model often scores respectably on IFEval and tool formats from pre-training alone — and spend the SFT budget on the goals where the gap is real. Chapter 11's pass-rate audit is the per-example version of the same rule.

23.4 Safety in SFT: teaching the model when to say no — and when not to

Safety behavior is installed in SFT before it is refined anywhere else, and it is the goal where the data's *balance* matters more than its size. The mechanism is the same as every other goal: show the model harmful requests paired with refusals, and it learns to refuse. The failure that follows is just as mechanical: it learns to refuse anything that *looks like* the harmful requests — the same lexical cues, the same topics — and a model that declines to explain how to kill a Python process, or what the mortality rate of a disease is, is the result. The XSTest suite made this measurable with 250 safe prompts that superficially resemble unsafe ones (homonyms, figurative language, historical questions, safe targets) alongside 200 unsafe contrasts, and found that the safety-tuned chat models of 2023 refused a large share of the safe prompts outright — over a third, for the most heavily tuned — while the models that answered them were not the ones that complied with the unsafe set[8]. Llama 2's own report measured the same trade-off from the inside: as the share of safety data in SFT grew, false refusals on ordinary

helpfulness prompts stayed negligible, but on a deliberately borderline set they rose to between 15% and 27%[9]. OR-Bench scaled the measurement to 80,000 "seemingly toxic" prompts and showed the pattern across model families: the models that refuse the most harmful prompts also refuse the most harmless ones, and moving along that frontier is a choice, not a free improvement[10].

The recipe that the open post-training reports converge on has four parts. **Pair every refusal with compliance.** For each category of harmful request in the data, include borderline and benign requests on the same topic with helpful answers — the "contrastive" set — so that the model learns the *boundary* rather than the topic. Tülu 3 used WildGuardMix and WildJailbreak for the harmful side and CoCoNot for the structure of noncompliance (requests that are unsafe, but also ones that are unanswerable, underspecified, or outside the model's abilities), each with its compliance counterpart[7]. **Keep the share small.** Safety data is a few percent of the SFT mix in every shipped recipe — a few thousand to a few tens of thousands of examples in a set of hundreds of thousands — because the norm generalizes from few examples and more of them only moves the boundary toward refusal. **Write the refusals well.** A refusal is an assistant turn like any other: it should name what it will not do, offer what it can, and not lecture; the model copies the register it is shown, and a dataset of curt refusals produces a curt model. **Measure both sides every time.** The harmful-request refusal rate alone is a vanity metric; report it next to XSTest or OR-Bench over-refusal, and treat a rise in the second as a regression equal to a fall in the first. Figure 23.3 draws the frontier.

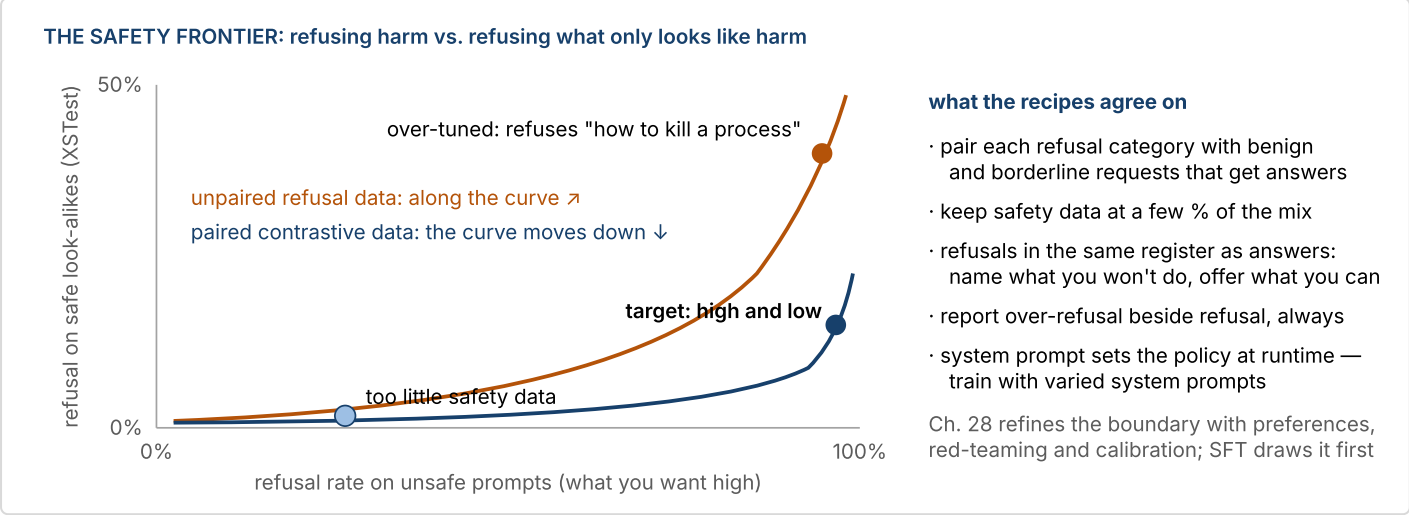


Figure 23.3 — Safety data moves you along a frontier, not up a ladder. Unpaired refusal examples raise both refusal rates together; paired contrastive examples are what push the frontier down. The measurement that keeps a team honest is the over-refusal number printed next to the refusal number.

Two further points belong here because they are decided in SFT. **The system prompt is the policy's runtime switch.** A deployment that needs different rules for different products (a children's tutor, a medical assistant, an internal tool) does not train three models; it trains one model to *obey its system prompt*, with SFT data in which varied system prompts set varied rules and the conversations honor them — and a model never shown system prompts during SFT will ignore them in production. **Language changes the boundary.** Refusal norms, legal content, and medical register differ between English and Latin American Spanish, and a safety set translated from English imports the wrong boundary; the contrastive pairs need to be written, or at least reviewed, in the target language, with Chapter 11's human sample as the audit. What SFT cannot do is make the boundary robust:

jailbreaks, multi-turn manipulation, and calibrated uncertainty are refined with preference data and adversarial testing in Chapter 28. SFT draws the line; the later stages defend it.

23.5 The arithmetic: FLOPs you can afford, memory you cannot

Chapter 8's accounting applies unchanged, and it says something reassuring about compute and something alarming about memory. **Compute** is $6N$ FLOPs per trained token (forward plus backward), counting every token in the sequence because the forward pass runs over the context even where the loss is masked. For the 7B/UltraChat example — 200K conversations averaging a thousand tokens, two epochs — that is

$$C_{\text{SFT}} = 6ND_{\text{SFT}} = 6 \times 7 \times 10^9 \times (2 \times 10^5 \cdot 10^3 \cdot 2) \approx 1.7 \times 10^{19} \text{ FLOPs},$$

about twelve H100-hours at the 40% utilization a well-packed SFT run reaches, against 66K GPU-hours for the pre-training run — a quarter of a percent. Tülu 3's full 940K-prompt mix at two epochs is closer to 2×10^{20} FLOPs, a long afternoon on eight GPUs; OpenThoughts3's 1.2M long traces at five epochs is the expensive end, in the hundreds of GPU-hours, because reasoning traces are ten thousand tokens each[4]. Packing (Chapter 16) is what makes the utilization real: conversations are short and variable, and padding a batch of them to 4,096 wastes most of the FLOPs you paid for.

Table 23.2 — A worked SFT budget for the 7B running example. Tokens = assistant + context tokens per example × examples × epochs; GPU-hours at 40% of an H100's dense peak (≈4×10^{14} FLOP/s); full fine-tuning at 6N FLOPs per token, LoRA at about two-thirds of that. Memory excludes activations (add 5–30 GB).

GOAL (FROM TABLE 23.1)	EXAMPLES × TOKENS × EPOCHS	TRAINING TOKENS	FULL FT	LORA	MEMORY FULL / LORA / QLORA
Format + register (LIMA-style)	1K × 800 × 3	2.4M	0.1 GPU-h	0.05 GPU-h	112 / 15 / 5 GB
Multi-turn chat	100K × 1,000 × 2	200M	6 GPU-h	4 GPU-h	112 / 15 / 5 GB
Instruction following + structured output + tools	60K × 900 × 2	108M	3 GPU-h	2 GPU-h	112 / 15 / 5 GB
Math + code (verified)	200K × 700 × 2	280M	8 GPU-h	5.5 GPU-h	112 / 15 / 5 GB
Long reasoning traces	100K × 8,000 × 3	2.4B	70 GPU-h	47 GPU-h	112 / 15 / 5 GB (+ large activations at 16K)
Safety (paired) + system prompts + locale	60K × 600 × 2	72M	2 GPU-h	1.3 GPU-h	112 / 15 / 5 GB
General assistant (all but long traces)	~420K examples	**~660M**	**~19 GPU-h**	**~13 GPU-h**	2× 80 GB / 1× 24 GB / 1× 16 GB
+ reasoning model	~520K examples	**~3.1B**	**~90 GPU-h**	**~60 GPU-h**	long context dominates; LoRA r ≥ 64 (Figure 23.5)

The table says where the money is. A general assistant's SFT is a day on one GPU with LoRA; adding long reasoning traces multiplies the token count by five, and the per-example length (not the example count) is what did it, which is why the reasoning recipes stage their long-trace SFT separately at long context and why the adapter's capacity (§23.6) becomes the binding question only on that row. Against the pre-training run's 66K GPU-hours, even the reasoning row is 0.14%.

Memory is where SFT stops being cheap, and the reason is the optimizer. Full fine-tuning in mixed precision with Adam holds, per parameter, a bf16 weight (2 bytes), a bf16 gradient (2), an fp32 master copy (4), and two fp32 Adam moments (8): 16 bytes, so a 7B model needs 112 GB before a single activation — two 80GB cards with sharding, or one card with CPU offload and a long wait. Figure 23.4 draws the bill against the two ways of shrinking it: freeze the weights and train small adapters, so the gradient and optimizer bytes exist only for the adapter's parameters (LoRA; the frozen weights stay in bf16, 14 GB for 7B), or additionally store the frozen

weights in 4 bits (QLoRA; 3.5 GB), which put a 65B model's fine-tuning on a single 48GB GPU and matched 16-bit full fine-tuning on the benchmarks the paper ran[11]. Activations are the remaining term and gradient checkpointing (Chapter 8) handles them at a third more compute.

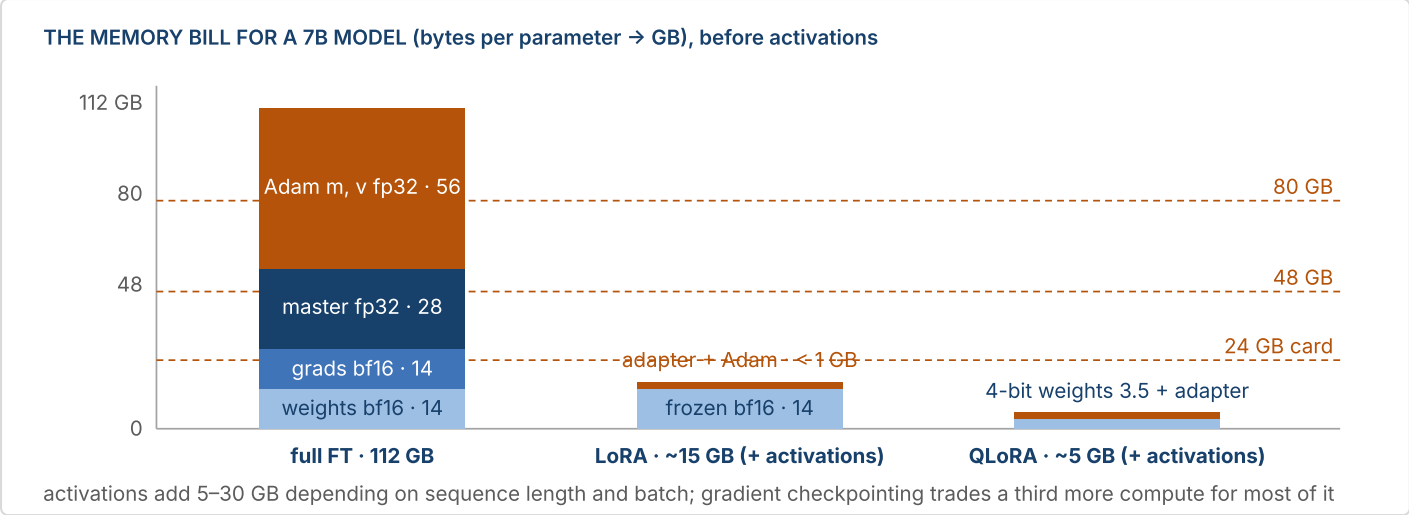


Figure 23.4 — Sixteen bytes per parameter is the problem; adapters remove twelve of them. Full fine-tuning's memory is dominated by the optimizer state for parameters that will barely move. Freezing the weights and training a low-rank adapter puts a 7B run on one 24–48GB card; 4-bit frozen weights put a 65B run on 48GB.

23.6 LoRA, done right

Low-rank adaptation freezes a weight matrix $W_0 \in R^{d \times k}$ and learns its update as a product of two thin matrices,

$$W = W_0 + \frac{\alpha}{r} BA, \qquad B \in R^{d \times r},\ A \in R^{r \times k},\ r \ll \min(d, k),$$

with A initialized randomly and B to zero so that training starts from the base model, and α/r a fixed scale that keeps the update's magnitude comparable across ranks. The trainable parameter count is $r(d + k)$ per matrix instead of dk: at rank 16 on a 4,096-wide layer that is 131K parameters against 16.8M, a 128× reduction, which is where the memory saving of Figure 23.4 comes from and why the original paper reported up to 10,000× fewer trainable parameters and 3× less GPU memory than full fine-tuning with no inference latency, since BA merges into W_0 after training[12]. The question since 2021 has been whether the constraint costs quality, and the answer in 2025 is precise enough to act on.

The careful comparison is a 2025 study that swept rank, learning rate, layer placement, and dataset size across Llama 3 and Qwen3 models on the Tülu 3 and OpenThoughts3 SFT sets and on RL for math[3]. Its findings are the recipe. **Apply LoRA to every linear layer, especially the MLPs.** The habit of adapting only attention projections is wrong: attention-only LoRA at rank 256 (0.25B trainable parameters) lost to MLP-only LoRA at rank 128 (0.24B), and adding attention adapters on top of MLP adapters bought nothing. **Use a learning rate about ten times the full-fine-tuning rate** — the fitted multiplier was 9.8 — and expect it to be nearly independent of rank: the optimum moved by less than 2× between rank 4 and rank 512. **Rank buys capacity, not speed.** Under the α/r scaling, learning curves are identical across ranks until the adapter fills up; a dataset that exceeds the adapter's

capacity trains less efficiently, and the gap grows with the ratio of data to adapter parameters. The capacity estimate that falls out is useful: a network stores on the order of 2 bits per parameter, and an SFT token carries about 1 bit of information at typical losses, so a rank-r adapter with P parameters can absorb a dataset of roughly $2P$ tokens before it is capacity-bound — tens of millions of tokens for a rank-16 adapter on a 7B model, which covers every SFT set in this chapter except Tülu 3's and OpenThoughts3's full mixes, where ranks of 128–256 or full fine-tuning are warranted. **Large batches hurt LoRA more than full fine-tuning**, and raising the rank does not fix it; keep the batch at 32–64 sequences. And **for RL, rank 1 is enough** — policy gradient absorbs about a bit per episode, far below any adapter's capacity — which Chapter 25 uses. Each LoRA step also costs about two-thirds of a full step's FLOPs, since the backward pass skips the weight gradients of frozen matrices.

The other side of the ledger is the *forgetting* result. On continued pre-training with 20B tokens of code or math, LoRA learned substantially less than full fine-tuning — the update a full run makes has a rank 10–100× higher than typical adapters — but it also forgot substantially less of the base model's other abilities, more so than weight decay or dropout achieved[13]. For SFT, where the information to learn is small and keeping the base's knowledge is the whole point, that trade is the one you want; for teaching a domain from 20B tokens it is not, and Chapter 20's continued pre-training is full-parameter for that reason. Figure 23.5 puts the decision on two axes.

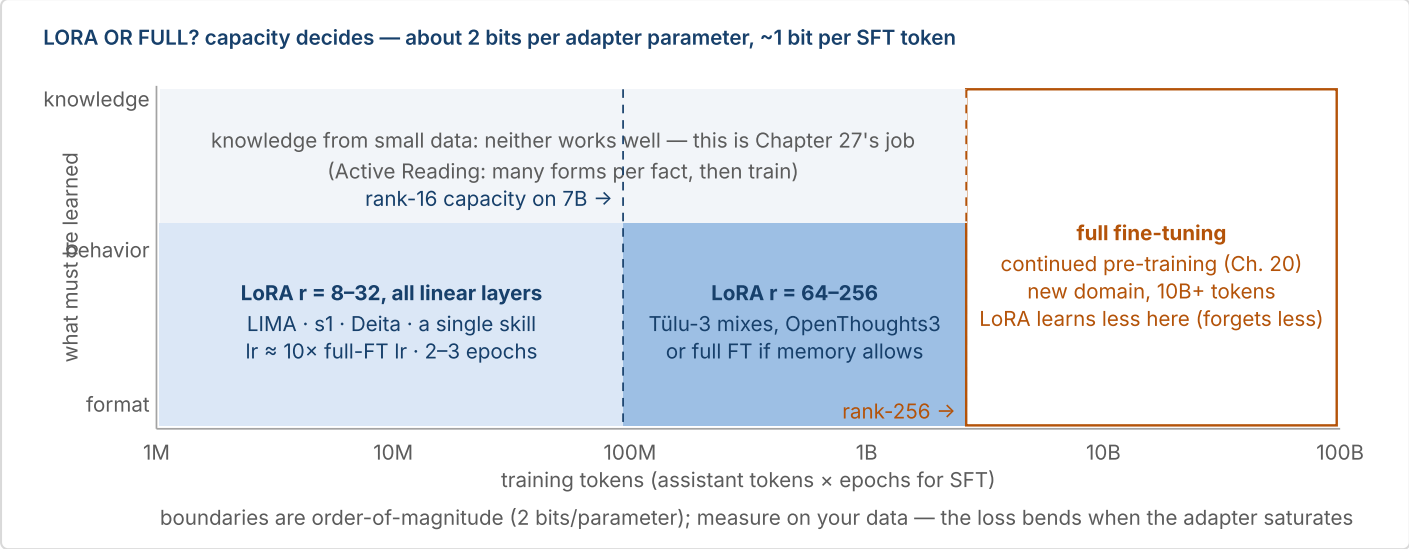


Figure 23.5 — Where LoRA is free and where it is not. Below the adapter's capacity, LoRA at the right learning rate matches full fine-tuning and forgets less; above it, training slows and full fine-tuning wins. SFT datasets live on the left; continued pre-training on the right; teaching knowledge from little data belongs to synthesis, not to either.

```
# LoRA that follows the evidence: all linear layers, rank by data size, lr ≈ 10× full
from peft import LoraConfig
lora = LoraConfig(
    r=16, lora_alpha=32,              # α/r = 2; r = 64–256 only for Tülu-3-size mix
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",          # attention …
                    "gate_proj", "up_proj", "down_proj"],            # … AND the MLPs
    lora_dropout=0.0, bias="none", task_type="CAUSAL_LM")
cfg = SFTConfig(output_dir="sft-7b-lora",
    learning_rate=1e-4,               # 10× the 1e-5 used for full fine-tuning
    per_device_train_batch_size=4, gradient_accumulation_steps=8,   # 32: keep it sma
    num_train_epochs=2, lr_scheduler_type="cosine", warmup_ratio=0.03,
    max_length=4096, packing=True, assistant_only_loss=True,
    bf16=True, gradient_checkpointing=True)
trainer = SFTTrainer(model="meta-llama/Llama-3.1-8B", train_dataset=ds, args=cfg,
                     peft_config=lora)
trainer.train()
trainer.model.merge_and_unload().save_pretrained("sft-7b-merged")   # no inference co
# QLoRA: load the base with BitsAndBytesConfig(load_in_4bit=True, quant_type="nf4",
#   bnb_4bit_compute_dtype=torch.bfloat16); everything else unchanged — 7B fits in ~1
```

23.7 The run: hyperparameters that transfer

SFT runs are short, so the hyperparameters that matter are few and the public recipes agree on them closely enough to copy. **Learning rate.** Full fine-tuning of a 7–8B model uses 5×10^{-6} to 2×10^{-5} with a cosine decay and 3% warm-up — Tülu 3 used 5×10^{-6} for its 8B SFT[7]; LoRA uses ten times that. Too high shows up within the first few hundred steps as a loss that drops and then climbs, and as a model that repeats itself. **Epochs.** Two for a general mix (Tülu 3), up to three for a small curated set, five for long reasoning traces where the model needs several passes to make the style its own (OpenThoughts3[4], s1[2]); the evaluation suite, not the training loss, says when to stop, because assistant-only loss keeps falling long after the model has stopped improving on anything you care about. **Sequence length.** Match the longest examples you keep: 4K for chat, 16–32K for reasoning traces, with packing so that the short conversations do not pay for the long ones. **Batch.** 32–128 sequences effective; LoRA at the low end. **Precision.** bf16 everywhere, with fp32 optimizer state in full fine-tuning.

Three refinements are cheap and real. **Mix in a slice of pre-training data** — 5–10% of the batch from the Chapter 10 corpus — when the SFT set is narrow; it is the same guardrail Chapter 27's Active Reading needed, and it costs nothing but a data loader. **Noise the embeddings.** NEFTune adds uniform noise to the input embeddings during SFT and, on single-turn instruction data, lifted a Llama-2-7B's AlpacaEval win rate from 29.8% to 64.7% with Alpaca and gave consistent gains on other sets, apparently by discouraging the model from memorizing exact phrasings[14]; it is a one-line flag and worth an ablation. **Train the end-of-turn token on purpose** and check that generation stops: a model that runs on is almost always a mask or template bug, not a hyperparameter.

23.8 Knowing it worked

The training loss of an SFT run tells you almost nothing after the first epoch, so the evaluation has to be in place before the run. Three instruments, in increasing cost. **Verifiable instruction following** — IFEval's 500-odd prompts with checkable constraints ("answer in exactly three bullet points," "do not use the letter e") — measures the format and compliance that SFT is supposed to teach, needs no judge, and is the first number to look at[15]. **Judged chat quality** — AlpacaEval in its length-controlled form, which removes the verbosity bias that made the original metric reward long answers[16], and Arena-Hard, built from hard real user prompts with a strong-model judge[17] — measures the sub-distribution choice, and carries every judge bias of Chapter 11 (verbosity, position, self-preference), so read the length-controlled number and keep a human sample. **The capability suite** of Chapter 9, run on the SFT model exactly as on the base, is the forgetting check: a drop on MMLU or GSM8K after SFT is the signal to mix in pre-training data, lower the learning rate, or move to LoRA.

Then the discipline from Chapter 11: a development suite that guides every data and hyperparameter choice, and an unseen suite you look at once, at the end, so that the gap between them tells you how much the recipe overfit the process[7]. The decision this evaluation feeds is when to stop SFT and start the next stage: when IFEval is high and stable, the forgetting check is flat, and the remaining failures are about *which* of several fluent answers is better — tone, length, honesty under uncertainty — you have reached what demonstrations can teach, and the preference and RL chapters take over.

23.9 Three recipes, step by step

Table 23.3 (§23.11) compresses eight reports to a row each; three of them are worth walking through, because each solved a different version of the problem and each is reproducible from public artifacts.

Tülu 3: a multi-skill mix, and how it was balanced

The Allen Institute's recipe is the most completely documented open post-training run, and its SFT stage is the template for "we need a general assistant." The target was a list of skills — knowledge recall, reasoning, math, code, precise instruction following, multilingual chat, safety — each with its own development evaluation. The mix of about 940K prompts was assembled source by source against those evaluations: public conversational data (WildChat, OpenAssistant, No Robots, FLAN v2) for general chat; persona-driven synthetic data (Chapter 27) generated specifically for the skills where public prompts were thin — Persona MATH, Persona GSM, Persona Python, Persona Algebra, and Persona IF for verifiable instruction following — in the tens to hundreds of thousands each; NuminaMath and Evol-CodeAlpaca for math and code; CoCoNot, WildGuardMix, and WildJailbreak for noncompliance and safety; Aya for multilingual coverage[7]. Every source was decontaminated against the evaluation suite with the released toolkit, and several public sets were dropped when contamination was found. The ablations are the lesson: removing any one skill's data lowered that skill's score and sometimes *raised* another's, so the final weights are a negotiated compromise rather than a maximum, and the paper publishes the per-source ablation table so you can see the trades. Training: full fine-tuning of Llama 3.1 8B at a learning rate of 5×10^{-6}, two epochs, 4K context, loss on assistant tokens only, followed by DPO and then RL with verifiable

rewards. The budget-conscious reading: the *structure* (skills → sources → per-skill evaluation → ablate → decontaminate) transfers to any team; the 940K count does not, and a team targeting three skills instead of ten needs a third of it.

SmolTalk: the same structure for a small model

The SmolLM2 recipe shows what changes when the student is 1.7B parameters and the goal is a capable small assistant[18]. The team found that the existing public mixes did not work well at that scale and built SmolTalk, about 1.1M examples, with a different center of gravity: roughly 400K multi-turn conversations generated with a Magpie-style method from a strong model (Smol-Magpie-Ultra), then purpose-built sets for the behaviors a small model most often gets wrong — Smol-constraints (verifiable instruction following), Smol-rewrite and Smol-summarize (editing and summarization with strict formats), SystemChats (system-prompt obedience), everyday conversations, and a function-calling set — alongside math (MetaMathQA, NuminaMath-CoT) and long-context examples. Two findings transfer. A small model needs *more* explicit, format-heavy data than a large one for the same goal, because less of the behavior is latent; and generated multi-turn data from a strong model was the single most valuable component, which is Chapter 27's format-gap argument made concrete. The hyperparameters are the usual ones at small scale: full fine-tuning, a learning rate around 3×10^{-4} for the 1.7B model, two epochs, 8K context, followed by DPO.

s1: selecting a thousand from fifty-nine thousand

The s1 recipe is the minimal-data extreme and its value is the *selection* procedure, which is Chapter 11 applied by hand[2]. The team collected about 59K reasoning questions from sixteen sources with traces generated by a strong model, then filtered in three passes. **Quality:** drop anything with API errors, formatting problems, or malformed traces (to about 52K). **Difficulty:** run two models on every question — Qwen2.5-7B-Instruct and Qwen2.5-32B-Instruct — and drop every question either of them solves, so that what remains is hard for the strongest available base, keeping trace length as a second difficulty signal (to about 25K). **Diversity:** classify the survivors into roughly fifty domains by subject and sample across them, favoring long traces, down to 1,000. Fine-tuning Qwen2.5-32B-Instruct on those 1,000 for five epochs produced s1-32B; the ablations showed that a random 1,000 or a diversity-only or difficulty-only 1,000 all did worse, and that training on the full 59K bought little over the selected 1,000. Add budget forcing at inference — appending "Wait" when the model tries to stop, to extend its reasoning — and AIME24 rose from 50% to 57%[2]. The transferable lesson is the difficulty pass: filtering by what your own strong model already solves is exactly Chapter 11's pass-rate audit, and it is what made a thousand examples enough.

Mixing and staging within SFT

Once the sources are chosen, two decisions remain that the recipes above make differently. **Weights and epochs per source.** A mixture is weighted by examples, but the sources differ in length by an order of magnitude (a chat turn of 200 tokens, a reasoning trace of 10,000), so weight by *tokens* when the goal is capability and by *examples* when the goal is behavior frequency — and consider per-source epoch counts: s1 and OpenThoughts3 ran five epochs over long traces while the general chat data in a mixed set saturates in two. **Staging.** A single mixed SFT

run is the default, but the reasoning-model recipes stage it: a general SFT first, then a long-trace SFT at longer context (DeepSeek-R1's cold-start then post-RL SFT; Qwen3's "long-CoT cold start" followed by general data later in the pipeline[19]). Staging lets each stage use its own context length and learning rate, and it keeps the long traces — the expensive tokens — out of the epochs that only need to teach chat. In either case the replay rule holds: a 5–10% slice of the base's pre-training data in every stage is the cheapest insurance against the forgetting check moving.

23.10 Multimodal SFT: the same objective with a second input

Vision-language models are post-trained with the same machinery and a change of input, and the pattern is worth knowing because it is now how most assistant models ship. The architecture is a frozen or lightly tuned vision encoder (a CLIP- or SigLIP-style ViT), a small *projector* that maps its patch features into the language model's embedding space, and the language model itself; an image becomes a few hundred to a few thousand "visual tokens" placed in the sequence where the image appears, and the loss is next-token cross-entropy on the assistant's text, exactly as in Figure 23.2 — the visual tokens are context, masked out like the user turn. LLaVA established the two-stage recipe: first train only the projector on image–caption pairs (595K filtered pairs) so that visual features land in the right space, then fine-tune the projector and the language model together on *visual instruction data* — conversations about an image, generated by prompting a strong text model with the image's captions and object boxes (158K examples); LLaVA-1.5 scaled the second stage to 665K examples mixing academic VQA datasets with the conversations and trained in about a day on eight A100s[20]. The frontier open models (Qwen2.5-VL, Llama 3.2 Vision) follow the same stages at far larger scale, with dynamic image resolution, video frames as more visual tokens, and document and chart data for OCR-heavy use[21].

What changes for the budget is the token count, not the method. An image at a thousand visual tokens makes every example several times longer than a text turn, so the FLOP arithmetic of §23.5 is dominated by the visual context, and packing is harder because images cannot be split; sequence lengths of 8–32K and gradient checkpointing are the norm. The data chapters apply with one addition: the faithfulness check of Chapter 27 is against the *image*, and a text-only judge cannot do it — so visual SFT sets are audited with a vision judge or by humans, and the hallucination benchmarks (object presence, counting) join the forgetting suite. Freezing the vision encoder and using LoRA on the language model is the parameter-efficient default and holds the base's text abilities while the projector and adapters learn to read pictures; unfreezing the encoder helps for documents and fine-grained domains at the cost of forgetting on text, which the forgetting suite will show. For a Spanish-first model the same locale rules apply, with one more: the OCR and document data should include Latin American forms, receipts, and signage, which the English-centric visual instruction sets lack almost entirely.

23.11 What the big labs do

Table 23.3 — SFT recipes, as disclosed. Data size, method, and the hyperparameters the reports give.

RECIPE	SFT DATA	METHOD AND HYPERPARAMETERS	WHAT IT SHOWS
LIMA[1]	1,000 curated pairs	Full FT of a 65B base; 15 epochs; no preference stage	Format and style from a thousand examples; the superficial alignment hypothesis
s1[2]	1,000 reasoning traces (s1K)	Full FT of Qwen2.5-32B-Instruct; 5 epochs; well under an hour on 16 H100s; budget forcing at inference	A behavior (long, checked reasoning) from a thousand traces
Tülu 3[7]	~940K prompts, many skills	Full FT; lr 5×10^{-6} (8B), 2 epochs, 4K context; then DPO and RLVR	The multi-skill mix; development vs. unseen evaluation
OpenThoughts3[4]	1.2M traces from QwQ-32B	Full FT of Qwen2.5-7B-Instruct; 5 epochs; long context	Reasoning SFT scales with data; 53% AIME 2025 at 7B
Llama 3[22]	Millions of rejection-sampled examples per round	Full FT; lr 10^{-5} at 8B; several SFT → DPO rounds; quality filtering by reward model	SFT as one stage of an iterated recipe
DeepSeek-R1[23]	~800K (600K reasoning + 200K general)	Cold-start SFT, RL, rejection sampling, SFT again, RL again	SFT twice: to give RL a format, and to consolidate what RL found
Guanaco / QLoRA[11]	OASST1, ~9K conversations	QLoRA on 7B–65B; 4-bit NF4 base, paged optimizers; 65B on one 48GB GPU	Parameter-efficient SFT matches 16-bit full FT on the benchmarks run
LoRA Without Regret[3]	Tülu 3, OpenThoughts3; GSM8K/MATH RL	LoRA on all layers, lr 10× full-FT, rank 1–512, Llama 3 / Qwen3	LoRA matches full FT below capacity; MLP layers matter; rank 1 suffices for RL

23.12 Practical considerations

Spend the week on data, the afternoon on training. The run is twelve GPU-hours; the Chapter 11 audits that decide what goes into it are where the quality is set. A 50K-example set that passed IFD, Min-K%, and decontamination beats a 500K download, and costs a tenth to train on.

Check the template and the mask before anything else. Decode one batch of labels. Confirm the chat template has generation markers so that `assistant_only_loss` applies. Confirm the end-of-turn token is a target. Confirm the same template is used at inference. These four checks catch most "the model won't stop" and "the model answers as the user" failures.

Default to LoRA on all linear layers; escalate deliberately. Rank 16, alpha 32, learning rate 10^{-4}, two epochs, batch 32 is the starting point for anything under a few hundred thousand examples. Watch the loss curve against a full-fine-tuning pilot on a 10% subset: if LoRA's curve bends away, the adapter is saturating and the rank goes up, or the run goes full.

Spanish and Portuguese. Use in-language SFT data (Chapter 27's persona-seeded and Active Reading outputs are the cheapest source), keep the chat template's special tokens language-neutral, evaluate with translated IFEval and a Spanish human sample, and watch the forgetting check on English — a monolingual SFT set pulls the model toward one language in a few hundred steps, and a 10% English slice in the batch holds it.

Common pitfalls

Training on the user turns.

The pre-training default. Half the gradient goes to predicting users, and the model learns to write the next question. Assistant-only loss, always.

LoRA on attention only, at the full-FT learning rate.

Both halves of the 2021 default are wrong by the 2025 evidence: the MLPs carry the adaptation, and LoRA wants ten times the learning rate.

Judging by training loss.

Assistant-only loss falls for epochs after the model stopped improving; IFEval and the forgetting suite decide when to stop.

Reading an uncontrolled win rate.

Longer answers win judged comparisons; use the length-controlled metric and read fifty comparisons yourself.

Using SFT to teach knowledge.

A thousand examples cannot carry facts; Figure 23.5's upper-left region is empty for a reason. Facts come from Chapter 27's many-forms synthesis or Chapter 20's continued pre-training.

Expecting SFT to generalize under shift.

It memorizes the training distribution; robustness comes from the RL stage, which needs SFT's format first.

Mismatched templates.

Trained with one chat template, served with another: fluent, wrong, and hard to diagnose. Pin the template with the tokenizer revision (Chapter 12).

23.13 Decision framework

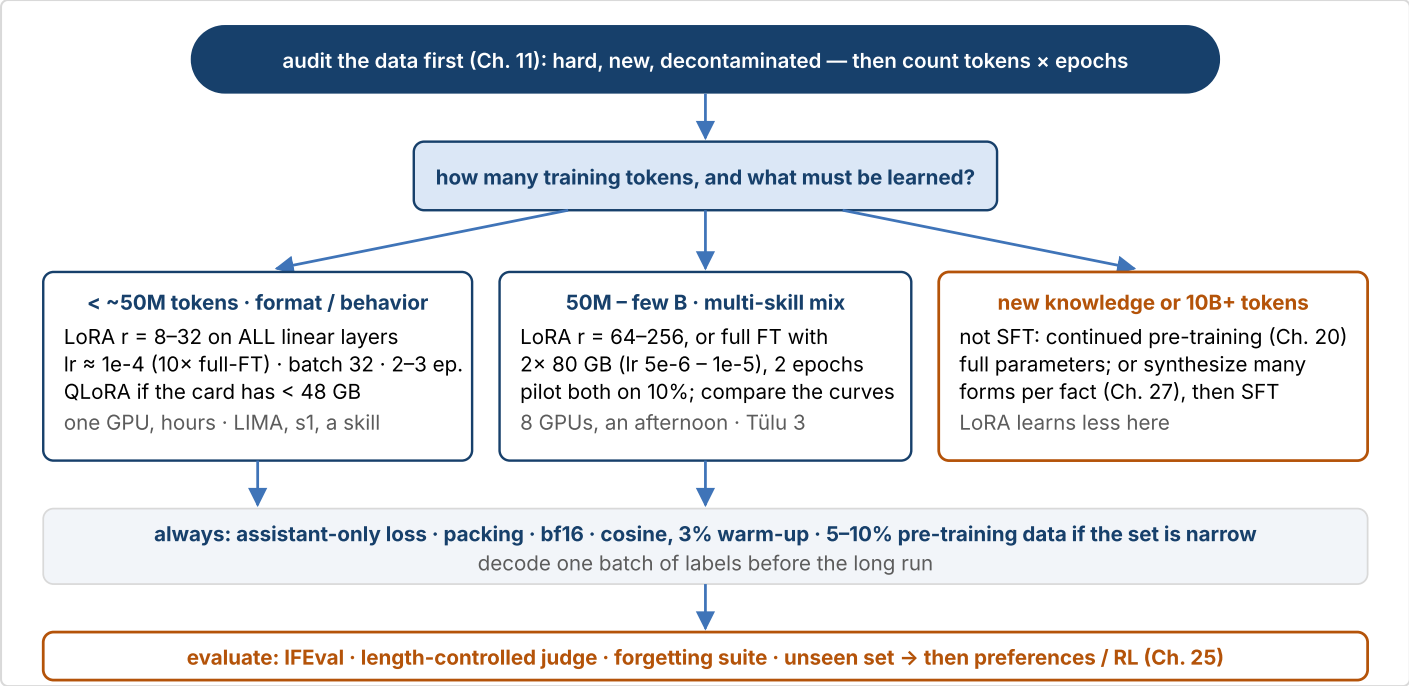


Figure 23.6 — Choosing the SFT recipe. Data audit first; the method follows from the token count and from what has to be learned; the loss mask, packing, and the evaluation ladder are the same on every path. The right-hand box is the reminder that SFT does not teach knowledge.

When it hurts. SFT hurts when the data is narrow and the learning rate high (the base's abilities slide in a few hundred steps — the forgetting suite catches it, and LoRA or a pre-training slice prevents it); when the set is large and the adapter small (the curve bends — raise the rank or go full); and when it is asked to do what it cannot — teach facts, or make the model robust to shifts it never saw.

How this scales. The SFT data budget does not grow with model size; the per-token cost does, linearly, and the memory bill grows with it — which is why LoRA's share of production SFT has risen with model size rather than fallen. At 70B, full fine-tuning needs over a terabyte of optimizer state across a node; LoRA needs the weights plus a few gigabytes, and the 2025 evidence says it loses nothing for the datasets SFT uses. The cheap version of SFT is the default version at every scale that matters.

23.14 Summary

Supervised fine-tuning adds a format and a choice of sub-distribution to a model that already has the knowledge, and sometimes a behavior that is visible in text; it does not add facts and it does not generalize under shift, which is why it is cheap and why it is followed by other stages. Its data is small, hard, and new — a thousand examples for a format, tens of thousands per skill for a mix, a million long traces for reasoning — and it is audited with Chapter 11's instruments before anything is trained. The objective is next-token cross-entropy on the assistant's tokens only, through the chat template the model will be served with, packed so that the FLOPs are real. The compute is a rounding error next to pre-training (twelve GPU-hours for a 7B model on 200K conversations); the

memory is not, because Adam's sixteen bytes per parameter put full fine-tuning of 7B on two 80GB cards — and low-rank adapters on all linear layers, at ten times the full-fine-tuning learning rate, remove twelve of those bytes while matching full fine-tuning for every dataset below the adapter's capacity, forgetting less of the base on the way. Hyperparameters transfer: 10^{-5} full or 10^{-4} LoRA, cosine, 2–3 epochs, batch 32–128, bf16. Judge the result with IFEval, a length-controlled judge, and the forgetting suite, not with the training loss, and stop when the remaining failures are about which fluent answer is better — the point where preferences and rewards take over.

References

1. Zhou, C., et al. (2023). LIMA: Less Is More for Alignment. *NeurIPS*. arXiv:2305.11206. 1,000 curated examples; outputs equivalent to or preferred over GPT-4's in 43% of comparisons; the superficial alignment hypothesis.
2. Muennighoff, N., et al. (2025). s1: Simple Test-Time Scaling. arXiv:2501.19393. s1K: 1,000 reasoning traces selected for difficulty, diversity, quality; SFT of Qwen2.5-32B-Instruct; up to 27% over o1-preview on MATH and AIME24; budget forcing.
3. Schulman, J., et al. (2025). LoRA Without Regret. Thinking Machines Lab blog, September 2025. LoRA matches full fine-tuning below capacity; all layers (MLP essential); optimal lr ≈ 10× full-FT (fitted 9.8), rank-independent; batch-size penalty; rank 1 suffices for RL; ~2 bits per parameter capacity; Llama 3 and Qwen3 on Tülu 3 and OpenThoughts3.
4. Guha, E., et al. (2025). OpenThoughts: Data Recipes for Reasoning Models. arXiv:2506.04178. 1.2M traces from QwQ-32B; OpenThinker3-7B at 53% AIME 2025; multi-epoch SFT on long traces.
5. Chu, T., et al. (2025). SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training. arXiv:2501.17161. GeneralPoints and V-IRL; SFT tracks training data under rule and visual shifts, RL transfers; SFT needed first to stabilize format.
6. Liu, W., et al. (2024). What Makes Good Data for Alignment? (Deita). *ICLR*. arXiv:2312.15685. 6K examples selected for complexity, quality, diversity match 10× more data.
7. Lambert, N., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. arXiv:2411.15124. ~940K-prompt SFT mixture; 8B SFT at lr 5×10^{-6}, 2 epochs; development vs. unseen evaluations; SFT → DPO → RLVR.
8. Röttger, P., et al. (2024). XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. *NAACL*. arXiv:2308.01263. 250 safe prompts of ten types that resemble unsafe ones, 200 unsafe contrasts; heavily safety-tuned chat models refused a large share of the safe prompts.
9. Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. Safety SFT and RLHF; false-refusal rate near zero on helpfulness prompts but 15–27% on a borderline set as safety data grew.
10. Cui, J., et al. (2024). OR-Bench: An Over-Refusal Benchmark for Large Language Models. arXiv:2405.20947. 80K seemingly toxic prompts across ten categories; models that refuse more harmful prompts also refuse more harmless ones.
11. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*. arXiv:2305.14314. 4-bit NF4 base weights, double quantization, paged optimizers; 65B fine-tuned on a single 48GB GPU; Guanaco.

12. Hu, E., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*. arXiv:2106.09685. Up to 10,000× fewer trainable parameters and 3× less GPU memory than full fine-tuning; no added inference latency after merging.
13. Biderman, D., et al. (2024). LoRA Learns Less and Forgets Less. *TMLR*. arXiv:2405.09673. Code and math, 100K-pair instruction tuning and 20B-token continued pre-training; LoRA underperforms full FT in learning but forgets less; full-FT updates have rank 10–100× typical LoRA ranks.
14. Jain, N., et al. (2024). NEFTune: Noisy Embeddings Improve Instruction Finetuning. *ICLR*. arXiv:2310.05914. Uniform noise on input embeddings; Llama-2-7B on Alpaca from 29.8% to 64.7% AlpacaEval.
15. Zhou, J., et al. (2023). Instruction-Following Evaluation for Large Language Models (IFEval). arXiv:2311.07911. ~500 prompts with verifiable constraints.
16. Dubois, Y., et al. (2024). Length-Controlled AlpacaEval: A Simple Way to Debias Automatic Evaluators. arXiv:2404.04475.
17. Li, T., et al. (2024). From Crowdsourced Data to High-Quality Benchmarks: Arena-Hard and BenchBuilder Pipeline. arXiv:2406.11939.
18. Allal, L. B., et al. (2025). SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model. arXiv:2502.02737. SmolTalk (~1.1M examples): Smol-Magpie-Ultra, Smol-constraints, Smol-rewrite, Smol-summarize, SystemChats, function calling; small models need more explicit format data.
19. Qwen Team (2025). Qwen3 Technical Report. arXiv:2505.09388. Staged post-training: long-CoT cold start, reasoning RL, thinking-mode fusion, general RL.
20. Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual Instruction Tuning (LLaVA). *NeurIPS*. arXiv:2304.08485. Projector pre-training on 595K image–caption pairs; 158K GPT-4-generated visual instruction examples. Liu, H., et al. (2024). Improved Baselines with Visual Instruction Tuning (LLaVA-1.5). *CVPR*. arXiv:2310.03744. 665K-example mix; about a day on 8 A100s.
21. Qwen Team (2025). Qwen2.5-VL Technical Report. arXiv:2502.13923. Dynamic-resolution vision encoder, document and video data in SFT. Llama 3.2 Vision follows the Llama 3 herd report (arXiv:2407.21783), adapter-based vision on a frozen language model.
22. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Iterated SFT and DPO rounds on rejection-sampled data; 8B SFT learning rate 10^{-5}.
23. DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. Cold-start SFT before RL; ~800K rejection-sampled examples for the second SFT stage.